

LAB 13: Transcriptomics and RNA-Seq Analysis

Kaif Yenepoya (A17497464)

Table of contents

Background	2
Data Import	2
Q1. How many genes are in this dataset?	3
Q2. How many control cell lines do we have?	3
Toy Differential Gene Expression	4
Q3. How would you make the above code more robust? Is there a function that could help here?	4
Q4. Follow the same procedure for the treated samples	4
Q5a. Create a scatter plot showing treated mean vs control mean	5
Q5b. What geom function would you use for this plot?	6
Q6. Try plotting both axes on a log scale. What is the argument to plot() that allows you to do this?	7
Q7. What is the purpose of arr.ind in which()? Why take the first column and use unique()?	9
Q8. How many up-regulated genes do we have at the greater than 2 fc level?	9
Q9. How many down-regulated genes do we have at the greater than 2 fc level? . . .	9
Q10. Do you trust these results? Why or why not?	10
Setting up for DESeq2	10
Principal Component Analysis	10
DESeq Analysis	12
Adding Annotation Data	15
Q11. Add Entrez ID, UniProt accession, and GENENAME	16
Volcano Plot - Data Visualization	18
Pathway Analysis	21
Q12. Plot the pathview figures for the top 2 down-regulated pathways	23
Optional: Plotting Counts for CRISPLD2	25
Session Information	28

Background

Today we are doing an RNA-seq analysis on airway smooth muscle cells treated with dexamethasone. We will compare control and treated samples, calculate gene expression changes, run DESeq2, add gene annotations, and make plots to visualize the results.

Data Import

Let's read the count data and metadata for this experiment.

```
counts <- read.csv("airway_scaledcounts.csv", row.names = 1)
metadata <- read.csv("airway_metadata.csv")
```

Have a wee peek:

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG00000000003	723	486	904	445	1170
ENSG00000000005	0	0	0	0	0
ENSG00000000419	467	523	616	371	582
ENSG00000000457	347	258	364	237	318
ENSG00000000460	96	81	73	66	118
ENSG00000000938	0	0	1	0	2

	SRR1039517	SRR1039520	SRR1039521
ENSG00000000003	1097	806	604
ENSG00000000005	0	0	0
ENSG00000000419	781	417	509
ENSG00000000457	447	330	324
ENSG00000000460	94	102	74
ENSG00000000938	0	0	0

And the metadata that tells us what is actually in the columns of our counts object:

```
head(metadata)
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866

```
4 SRR1039513 treated N052611 GSM1275867
5 SRR1039516 control N080611 GSM1275870
6 SRR1039517 treated N080611 GSM1275871
```

```
metadata
```

```
      id      dex celltype    geo_id
1 SRR1039508 control  N61311 GSM1275862
2 SRR1039509 treated  N61311 GSM1275863
3 SRR1039512 control  N052611 GSM1275866
4 SRR1039513 treated  N052611 GSM1275867
5 SRR1039516 control  N080611 GSM1275870
6 SRR1039517 treated  N080611 GSM1275871
7 SRR1039520 control  N061011 GSM1275874
8 SRR1039521 treated  N061011 GSM1275875
```

Q1. How many genes are in this dataset?

```
nrow(counts)
```

```
[1] 38694
```

There are 38694 genes in this dataset.

Q2. How many control cell lines do we have?

```
sum(metadata$dex == "control")
```

```
[1] 4
```

There are 4 control cell lines.

```
table(metadata$dex)
```

```
control treated
      4      4
```

Check that the column names of `counts` match the sample IDs in `metadata`.

```
colnames(counts) == metadata$id
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
all(colnames(counts) == metadata$id)
```

```
[1] TRUE
```

Toy Differential Gene Expression

Find the control columns in our counts object.

```
control <- metadata[metadata$dex == "control", ]
control.counts <- counts[, control$id]
control.mean <- rowMeans(control.counts)
head(control.mean)
```

```
ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
          900.75           0.00           520.50           339.75           97.25
ENSG000000000938
          0.75
```

Q3. How would you make the above code more robust? Is there a function that could help here?

I would use `rowMeans()` instead of manually dividing by 4. This is more robust because it still works if the number of control samples changes.

Q4. Follow the same procedure for the treated samples

Now do the same for the treated columns.

```
treated <- metadata[metadata$dex == "treated", ]
treated.counts <- counts[, treated$id]
treated.mean <- rowMeans(treated.counts)
head(treated.mean)
```

```

ENSG000000000003  ENSG000000000005  ENSG000000000419  ENSG000000000457  ENSG000000000460
      658.00           0.00           546.00           316.50           78.75
ENSG0000000000938
      0.00

```

Now combine the mean counts into one data frame.

```

meancounts <- data.frame(control.mean, treated.mean)
head(meancounts)

```

```

      control.mean treated.mean
ENSG000000000003    900.75     658.00
ENSG000000000005      0.00      0.00
ENSG000000000419    520.50     546.00
ENSG000000000457    339.75     316.50
ENSG000000000460     97.25      78.75
ENSG000000000938      0.75      0.00

```

Check the sum of mean counts across all genes.

```

colSums(meancounts)

```

```

control.mean treated.mean
  23005324    22196524

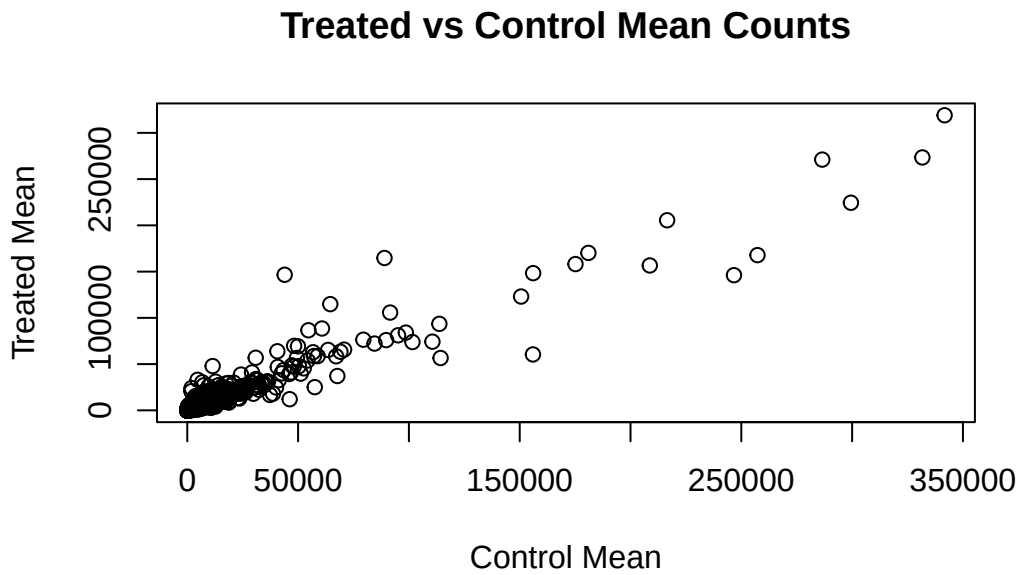
```

Q5a. Create a scatter plot showing treated mean vs control mean

```

plot(meancounts$control.mean, meancounts$treated.mean,
     xlab = "Control Mean",
     ylab = "Treated Mean",
     main = "Treated vs Control Mean Counts")

```

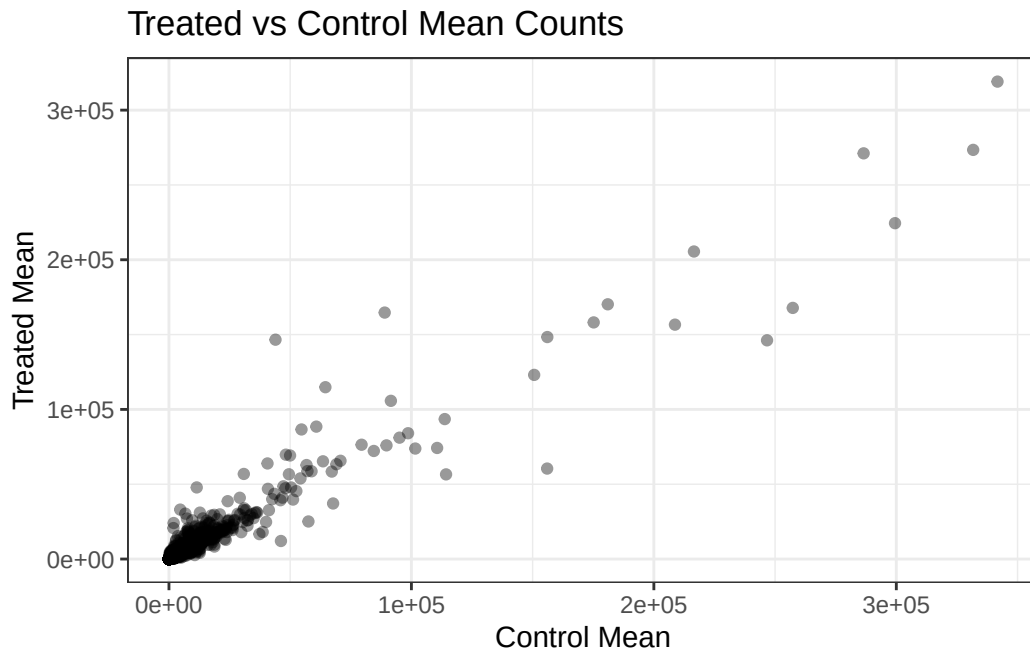


Q5b. What geom function would you use for this plot?

I would use `geom_point()` because this is a scatter plot.

```
library(ggplot2)

ggplot(meancounts, aes(x = control.mean, y = treated.mean)) +
  geom_point(alpha = 0.4) +
  labs(x = "Control Mean",
       y = "Treated Mean",
       title = "Treated vs Control Mean Counts") +
  theme_bw()
```



Q6. Try plotting both axes on a log scale. What is the argument to `plot()` that allows you to do this?

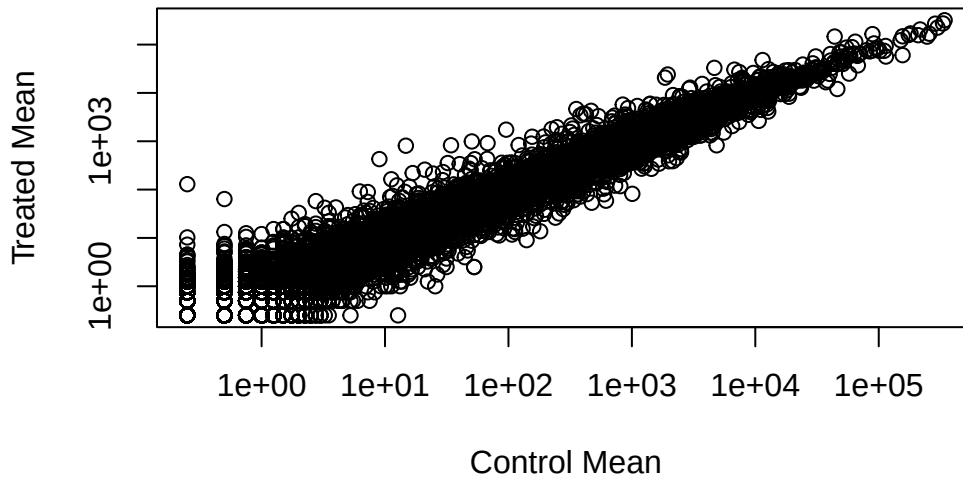
The argument is `log = "xy"` because it puts both the x-axis and y-axis on a log scale.

```
plot(meancounts$control.mean, meancounts$treated.mean,  
     log = "xy",  
     xlab = "Control Mean",  
     ylab = "Treated Mean",  
     main = "Treated vs Control Mean Counts on Log Scale")
```

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15032 x values ≤ 0 omitted from logarithmic plot

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15281 y values ≤ 0 omitted from logarithmic plot

Treated vs Control Mean Counts on Log Scale



Now calculate log2 fold change.

```
meancounts$log2fc <- log2(meancounts[, "treated.mean"] / meancounts[, "control.mean"])
head(meancounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG0000000000419	520.50	546.00	0.06900279
ENSG0000000000457	339.75	316.50	-0.10226805
ENSG0000000000460	97.25	78.75	-0.30441833
ENSG0000000000938	0.75	0.00	-Inf

Remove genes with zero values because they can create NaN, Inf, or -Inf.

```
zero.vals <- which(meancounts[, 1:2] == 0, arr.ind = TRUE)

to.rm <- unique(zero.vals[, 1])
mycounts <- meancounts[-to.rm, ]

head(mycounts)
```


	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000971	5219.00	6687.50	0.35769358
ENSG000000001036	2327.00	1785.75	-0.38194109

Q7. What is the purpose of `arr.ind` in `which()`? Why take the first column and use `unique()`?

The `arr.ind = TRUE` argument makes `which()` return the row and column positions of the zero values. We take the first column because it gives the row numbers, and `unique()` makes sure each gene is only removed once.

Now find genes above the fold-change thresholds.

```
up.ind <- mycounts$log2fc > 2
down.ind <- mycounts$log2fc < -2
```

Q8. How many up-regulated genes do we have at the greater than 2 fc level?

```
sum(up.ind)
```

```
[1] 250
```

There are 250 up-regulated genes with log2 fold change greater than 2.

Q9. How many down-regulated genes do we have at the greater than 2 fc level?

```
sum(down.ind)
```

```
[1] 367
```

There are 367 down-regulated genes with log2 fold change less than -2.

Q10. Do you trust these results? Why or why not?

I do not fully trust these results because this is only a simple raw count comparison. It does not properly account for sequencing depth, variation between samples, or statistical significance, so DESeq2 is needed for a better analysis.

Setting up for DESeq2

Load DESeq2.

```
library(DESeq2)
```

Create the DESeqDataSet object.

```
dds <- DESeqDataSetFromMatrix(countData = counts,  
                              colData = metadata,  
                              design = ~ dex)
```

converting counts to integer mode

```
dds
```

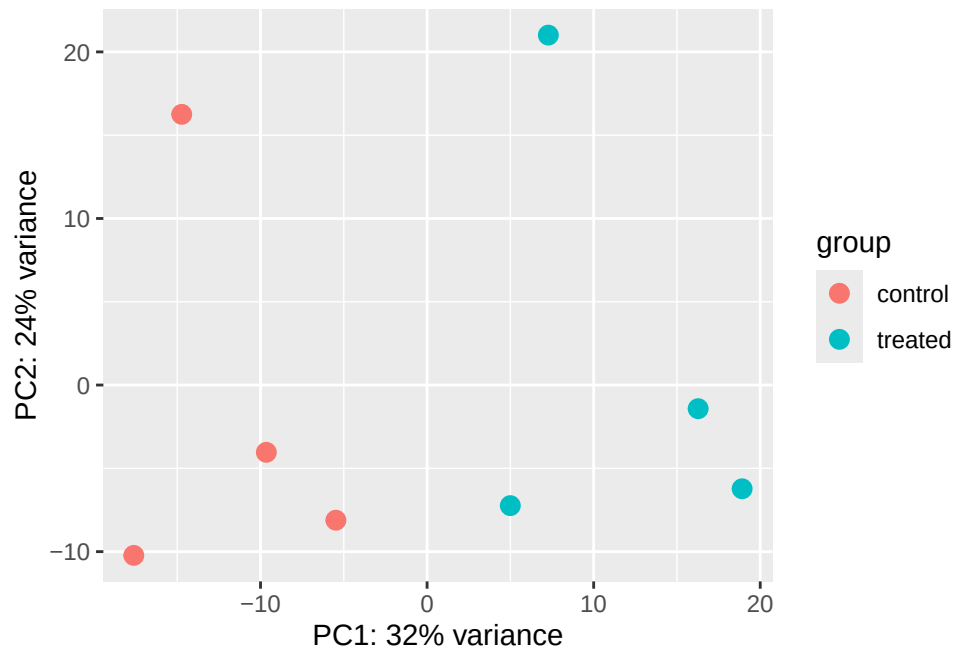
```
class: DESeqDataSet  
dim: 38694 8  
metadata(1): version  
assays(1): counts  
rownames(38694): ENSG000000000003 ENSG000000000005 ... ENSG00000283120  
               ENSG00000283123  
rowData names(0):  
colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521  
colData names(4): id dex celltype geo_id
```

Principal Component Analysis

We can use PCA to see how the samples group before running the full DESeq analysis.

```
vsd <- vst(dds, blind = FALSE)  
plotPCA(vsd, intgroup = c("dex"))
```

using ntop=500 top features by variance



Make the PCA plot using ggplot2.

```
pcaData <- plotPCA(vsd, intgroup = c("dex"), returnData = TRUE)
```

using ntop=500 top features by variance

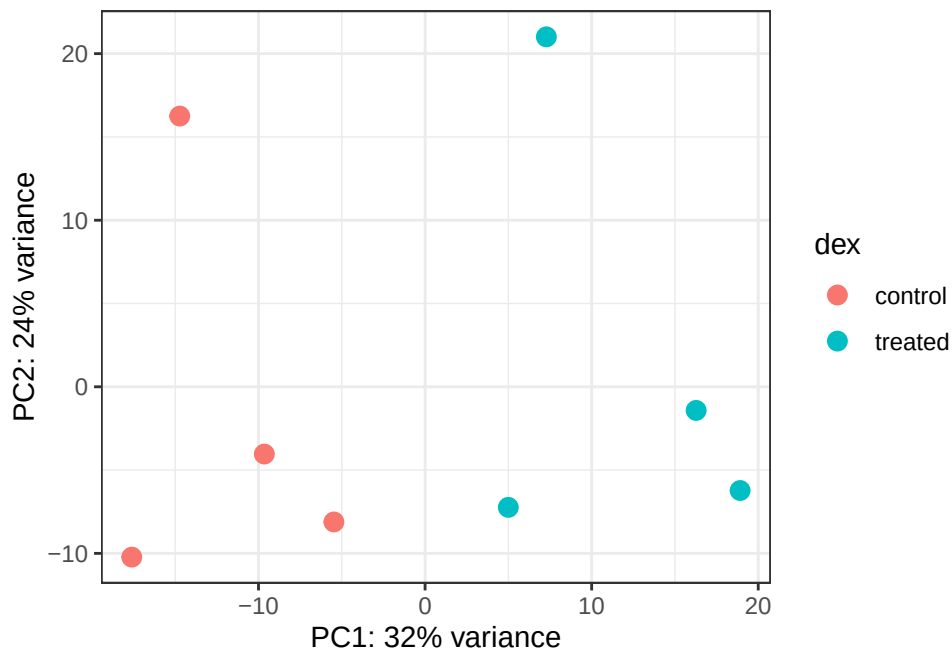
```
head(pcaData)
```

	PC1	PC2	group	name	id	dex	celltype
SRR1039508	-17.607922	-10.225252	control	SRR1039508	SRR1039508	control	N61311
SRR1039509	4.996738	-7.238117	treated	SRR1039509	SRR1039509	treated	N61311
SRR1039512	-5.474456	-8.113993	control	SRR1039512	SRR1039512	control	N052611
SRR1039513	18.912974	-6.226041	treated	SRR1039513	SRR1039513	treated	N052611
SRR1039516	-14.729173	16.252000	control	SRR1039516	SRR1039516	control	N080611
SRR1039517	7.279863	21.008034	treated	SRR1039517	SRR1039517	treated	N080611
	geo_id	sizeFactor					
SRR1039508	GSM1275862	1.0193796					
SRR1039509	GSM1275863	0.9005653					
SRR1039512	GSM1275866	1.1784239					

```
SRR1039513 GSM1275867 0.6709854
SRR1039516 GSM1275870 1.1731984
SRR1039517 GSM1275871 1.3929361
```

```
percentVar <- round(100 * attr(pcaData, "percentVar"))

ggplot(pcaData) +
  aes(x = PC1, y = PC2, color = dex) +
  geom_point(size = 3) +
  xlab(paste0("PC1: ", percentVar[1], "% variance")) +
  ylab(paste0("PC2: ", percentVar[2], "% variance")) +
  coord_fixed() +
  theme_bw()
```



DESeq Analysis

Run the DESeq pipeline.

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
res <- results(dds)
res
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 38694 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.1942	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.0000	NA	NA	NA	NA
ENSG000000000419	520.1342	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.6648	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.6826	-0.147143	0.256995	-0.572550	0.5669497
...	...	...	...	...	...
ENSG00000283115	0.000000	NA	NA	NA	NA
ENSG00000283116	0.000000	NA	NA	NA	NA
ENSG00000283119	0.000000	NA	NA	NA	NA
ENSG00000283120	0.974916	-0.66825	1.69441	-0.394385	0.693297
ENSG00000283123	0.000000	NA	NA	NA	NA
	padj				
	<numeric>				
ENSG000000000003	0.163017				
ENSG000000000005	NA				
ENSG000000000419	0.175937				
ENSG000000000457	0.961682				
ENSG000000000460	0.815805				
...	...				
ENSG00000283115	NA				
ENSG00000283116	NA				
ENSG00000283119	NA				
ENSG00000283120	NA				
ENSG00000283123	NA				

Convert the results to a data frame and preview it.

```
res.df <- as.data.frame(res)
head(res.df)
```

	baseMean	log2FoldChange	lfcSE	stat	pvalue
ENSG000000000003	747.1941954	-0.35070296	0.1682421	-2.0845139	0.03711345
ENSG000000000005	0.0000000	NA	NA	NA	NA
ENSG000000000419	520.1341601	0.20610728	0.1010415	2.0398279	0.04136747
ENSG000000000457	322.6648439	0.02452701	0.1451339	0.1689958	0.86579996
ENSG000000000460	87.6826252	-0.14714263	0.2569954	-0.5725496	0.56694971
ENSG000000000938	0.3191666	-1.73228897	3.4936010	-0.4958463	0.62000288
	padj				
ENSG000000000003	0.1630172				
ENSG000000000005	NA				
ENSG000000000419	0.1759366				
ENSG000000000457	0.9616825				
ENSG000000000460	0.8158052				
ENSG000000000938	NA				

Summarize the DESeq results.

```
summary(res)
```

```
out of 25258 with nonzero total read count
adjusted p-value < 0.1
LFC > 0 (up)      : 1564, 6.2%
LFC < 0 (down)    : 1188, 4.7%
outliers [1]      : 142, 0.56%
low counts [2]    : 9971, 39%
(mean count < 10)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

Use an adjusted p-value cutoff of 0.05.

```
res05 <- results(dds, alpha = 0.05)
summary(res05)
```

```

out of 25258 with nonzero total read count
adjusted p-value < 0.05
LFC > 0 (up)      : 1237, 4.9%
LFC < 0 (down)    : 933, 3.7%
outliers [1]      : 142, 0.56%
low counts [2]    : 9033, 36%
(mean count < 6)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results

```

Many adjusted p-values are missing because DESeq2 filters out genes with very low counts. These genes usually do not have enough expression to give reliable statistical evidence, so filtering improves the power of the analysis.

Adding Annotation Data

Load annotation packages.

```

library(AnnotationDbi)
library(org.Hs.eg.db)

```

Check available annotation columns.

```

columns(org.Hs.eg.db)

```

```

[1] "ACCNUM"      "ALIAS"       "ENSEMBL"     "ENSEMBLPROT" "ENSEMBLTRANS"
[6] "ENTREZID"    "ENZYME"      "EVIDENCE"    "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"    "GO"          "GOALL"       "IPI"          "MAP"
[16] "OMIM"        "ONTOLOGY"    "ONTOLOGYALL" "PATH"         "PFAM"
[21] "PMID"        "PROSITE"     "REFSEQ"      "SYMBOL"       "UCSCCKG"
[26] "UNIPROT"

```

Add gene symbols.

```

res$symbol <- mapIds(org.Hs.eg.db,
                      keys = row.names(res),
                      keytype = "ENSEMBL",
                      column = "SYMBOL",
                      multiVals = "first")

```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 7 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG0000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG0000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG0000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG0000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029
	padj	symbol			
	<numeric>	<character>			
ENSG0000000000003	0.163017	TSPAN6			
ENSG0000000000005	NA	TNMD			
ENSG00000000000419	0.175937	DPM1			
ENSG00000000000457	0.961682	SCYL3			
ENSG00000000000460	0.815805	FIRRM			
ENSG00000000000938	NA	FGR			

Q11. Add Entrez ID, UniProt accession, and GENENAME

```
res$entrez <- mapIds(org.Hs.eg.db,  
  keys = row.names(res),  
  keytype = "ENSEMBL",  
  column = "ENTREZID",  
  multiVals = "first")
```

'select()' returned 1:many mapping between keys and columns

```
res$uniprot <- mapIds(org.Hs.eg.db,  
  keys = row.names(res),  
  keytype = "ENSEMBL",  
  column = "UNIPROT",  
  multiVals = "first")
```


'select()' returned 1:many mapping between keys and columns

```
res$genename <- mapIds(org.Hs.eg.db,  
  keys = row.names(res),  
  keytype = "ENSEMBL",  
  column = "GENENAME",  
  multiVals = "first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG00000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG00000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029
	padj	symbol	entrez	uniprot	
	<numeric>	<character>	<character>	<character>	
ENSG00000000003	0.163017	TSPAN6	7105	AOA087WYV6	
ENSG00000000005	NA	TNMD	64102	Q9H2S6	
ENSG000000000419	0.175937	DPM1	8813	H0Y368	
ENSG000000000457	0.961682	SCYL3	57147	X6RHX1	
ENSG000000000460	0.815805	FIRRM	55732	A6NFP1	
ENSG000000000938	NA	FGR	2268	B7Z6W7	
	genename				
	<character>				
ENSG00000000003	tetraspanin 6				
ENSG00000000005	tenomodulin				
ENSG000000000419	dolichyl-phosphate m..				
ENSG000000000457	SCY1 like pseudokina..				
ENSG000000000460	FIGNL1 interacting r..				
ENSG000000000938	FGR proto-oncogene, ..				

Order the results by adjusted p-value.

```
ord <- order(res$padj)
head(res[ord, ])
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG00000152583	954.771	4.36836	0.2371306	18.4217	8.79214e-76
ENSG00000179094	743.253	2.86389	0.1755659	16.3123	8.06568e-60
ENSG00000116584	2277.913	-1.03470	0.0650826	-15.8983	6.51317e-57
ENSG00000189221	2383.754	3.34154	0.2124091	15.7316	9.17960e-56
ENSG00000120129	3440.704	2.96521	0.2036978	14.5569	5.27883e-48
ENSG00000148175	13493.920	1.42717	0.1003811	14.2175	7.13625e-46

	padj	symbol	entrez	uniprot
	<numeric>	<character>	<character>	<character>
ENSG00000152583	1.33157e-71	SPARCL1	8404	B4E2Z0
ENSG00000179094	6.10774e-56	PER1	5187	A2I2P6
ENSG00000116584	3.28806e-53	ARHGEF2	9181	A0A8Q3SIN5
ENSG00000189221	3.47563e-52	MAOA	4128	B4DF46
ENSG00000120129	1.59896e-44	DUSP1	1843	B4DRR4
ENSG00000148175	1.80131e-42	STOM	2040	F8VSL7

	genename
	<character>
ENSG00000152583	SPARC like 1
ENSG00000179094	period circadian reg..
ENSG00000116584	Rho/Rac guanine nucl..
ENSG00000189221	monoamine oxidase A
ENSG00000120129	dual specificity pho..
ENSG00000148175	stomatin

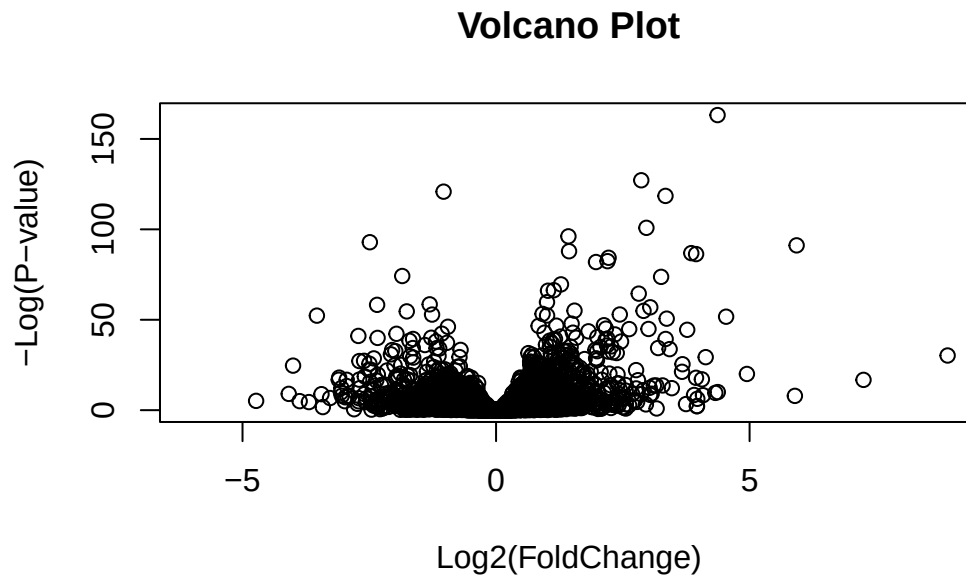
Write the ordered results to a CSV file.

```
write.csv(res[ord, ], "deseq_results.csv")
```

Volcano Plot - Data Visualization

Make a basic volcano plot.

```
plot(res$log2FoldChange, -log(res$padj),
     xlab = "Log2(FoldChange)",
     ylab = "-Log(P-value)",
     main = "Volcano Plot")
```



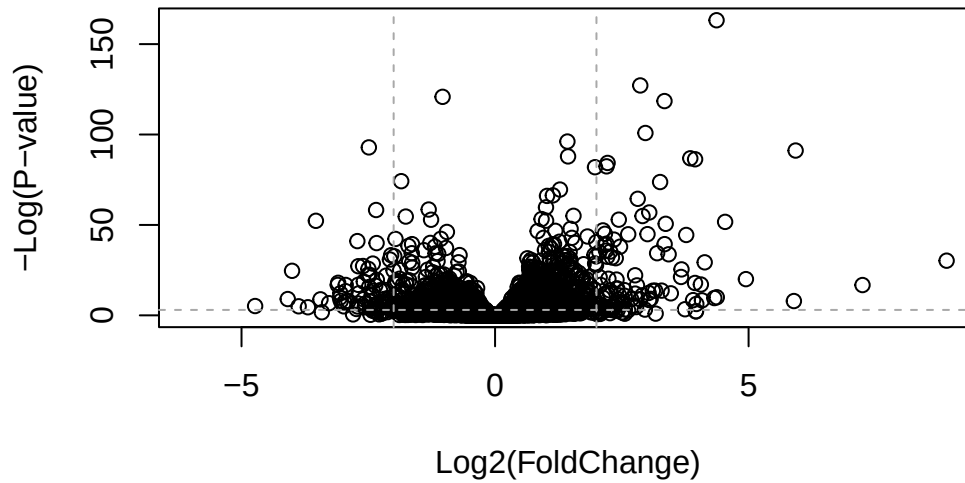
This plot is not very useful because we care most about genes with very low adjusted p-values.

Now add cutoff lines for log2 fold change and adjusted p-value.

```
plot(res$log2FoldChange, -log(res$padj),
     xlab = "Log2(FoldChange)",
     ylab = "-Log(P-value)",
     main = "Volcano Plot with Cutoffs")

abline(v = c(-2, 2), col = "darkgray", lty = 2)
abline(h = -log(0.05), col = "darkgray", lty = 2)
```

Volcano Plot with Cutoffs



Color the genes that meet the thresholds.

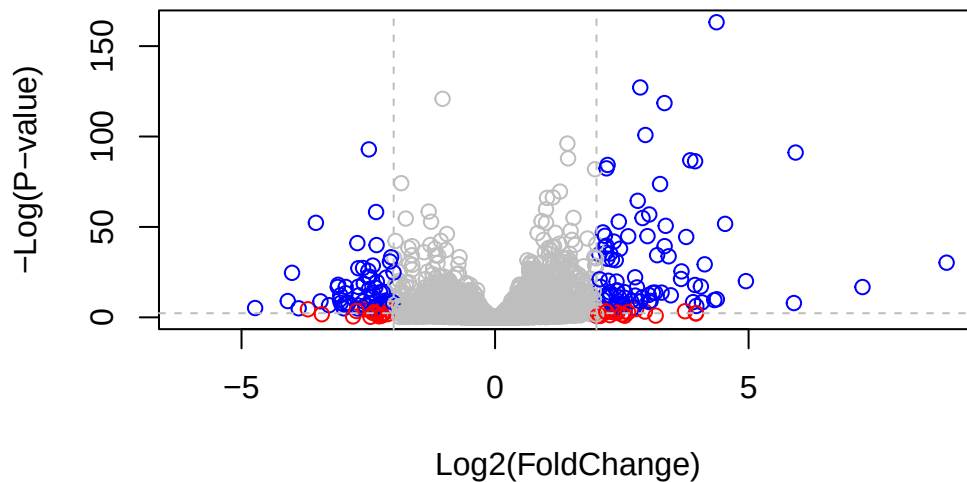
```
mycols <- rep("gray", nrow(res))
mycols[abs(res$log2FoldChange) > 2] <- "red"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2)
mycols[inds] <- "blue"

plot(res$log2FoldChange, -log(res$padj),
     col = mycols,
     xlab = "Log2(FoldChange)",
     ylab = "-Log(P-value)",
     main = "Colored Volcano Plot")

abline(v = c(-2, 2), col = "gray", lty = 2)
abline(h = -log(0.1), col = "gray", lty = 2)
```

Colored Volcano Plot



The volcano plot helps show genes with both large fold changes and strong statistical significance. The blue points are the most important genes because they pass both the fold-change and p-value cutoffs.

Pathway Analysis

Load the pathway analysis packages.

```
library(pathview)
library(gage)
library(gageData)
```

Load the KEGG pathway data.

```
data(kegg.sets.hs)
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
[1] "10"    "1544" "1548" "1549" "1553" "7498" "9"
```

```
$`hsa00983 Drug metabolism - other enzymes`
```

```

[1] "10"      "1066"    "10720"   "10941"   "151531"  "1548"    "1549"    "1551"
[9] "1553"    "1576"    "1577"    "1806"    "1807"    "1890"    "221223"  "2990"
[17] "3251"    "3614"    "3615"    "3704"    "51733"   "54490"   "54575"   "54576"
[25] "54577"   "54578"   "54579"   "54600"   "54657"   "54658"   "54659"   "54963"
[33] "574537"  "64816"   "7083"    "7084"    "7172"    "7363"    "7364"    "7365"
[41] "7366"    "7367"    "7371"    "7372"    "7378"    "7498"    "79799"   "83549"
[49] "8824"    "8833"    "9"       "978"

```

Create a named vector of fold changes.

```

foldchanges <- res$log2FoldChange
names(foldchanges) <- res$entrez

head(foldchanges)

```

```

      7105      64102      8813      57147      55732      2268
-0.35070296      NA  0.20610728  0.02452701 -0.14714263 -1.73228897

```

Run the GAGE pathway analysis.

```

keggres <- gage(foldchanges, gsets = kegg.sets.hs)

attributes(keggres)

```

```

$names
[1] "greater" "less"    "stats"

```

Look at the first few down-regulated pathways.

```

head(keggres$less, 3)

```

		p.geomean	stat.mean	p.val
hsa05332	Graft-versus-host disease	0.0004250607	-3.473335	0.0004250607
hsa04940	Type I diabetes mellitus	0.0017820379	-3.002350	0.0017820379
hsa05310	Asthma	0.0020046180	-3.009045	0.0020046180

		q.val	set.size	exp1
hsa05332	Graft-versus-host disease	0.09053792	40	0.0004250607
hsa04940	Type I diabetes mellitus	0.14232788	42	0.0017820379
hsa05310	Asthma	0.14232788	29	0.0020046180

Look at the first few up-regulated pathways.

```
head(keggres$greater, 3)
```

		p.geomean	stat.mean	p.val
hsa00500	Starch and sucrose metabolism	0.00330618	2.772653	0.00330618
hsa00330	Arginine and proline metabolism	0.01524126	2.194146	0.01524126
hsa04910	Insulin signaling pathway	0.01711093	2.129512	0.01711093
		q.val	set.size	exp1
hsa00500	Starch and sucrose metabolism	0.7042163	52	0.00330618
hsa00330	Arginine and proline metabolism	0.7774871	53	0.01524126
hsa04910	Insulin signaling pathway	0.7774871	138	0.01711093

Plot the asthma pathway.

```
pathview(gene.data = foldchanges, pathway.id = "hsa05310")
```

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory /Users/kaifyenepoya/Desktop/BIMM143/LAB_13

Info: Writing image file hsa05310.pathview.png

Q12. Plot the pathview figures for the top 2 down-regulated pathways

Get the top two down-regulated pathways.

```
top2.down <- rownames(keggres$less)[1:2]
top2.down
```

```
[1] "hsa05332 Graft-versus-host disease" "hsa04940 Type I diabetes mellitus"
```

Extract the KEGG pathway IDs.

```
top2.ids <- substr(top2.down, 1, 8)
top2.ids
```

```
[1] "hsa05332" "hsa04940"
```

Make pathview plots for the top two down-regulated pathways.

```
pathview(gene.data = foldchanges, pathway.id = top2.ids[1])
```

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory /Users/kaifyenepoya/Desktop/BIMM143/LAB_13

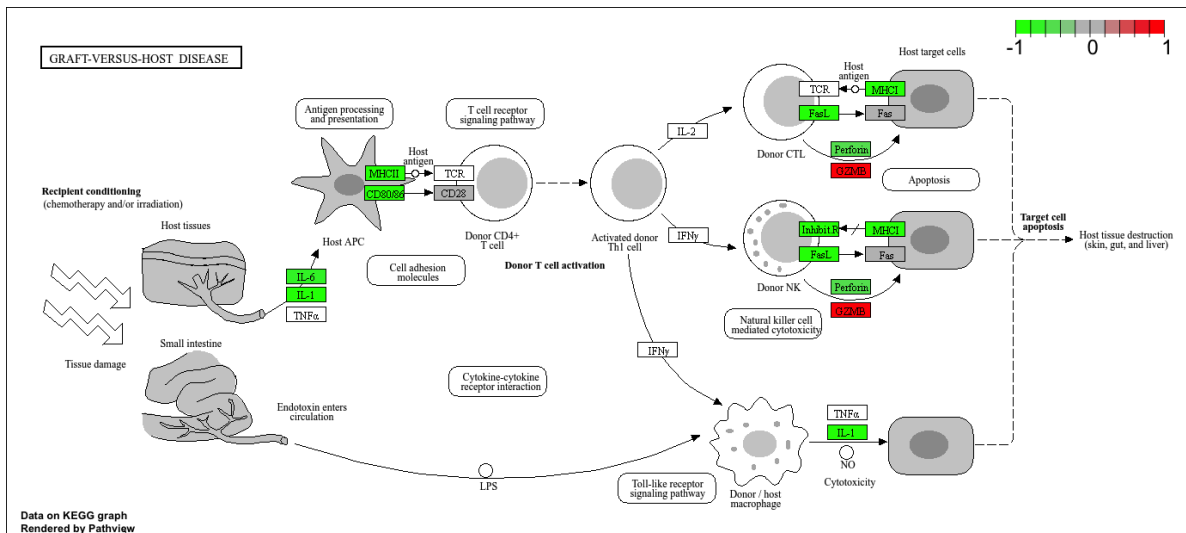
Info: Writing image file hsa05332.pathview.png

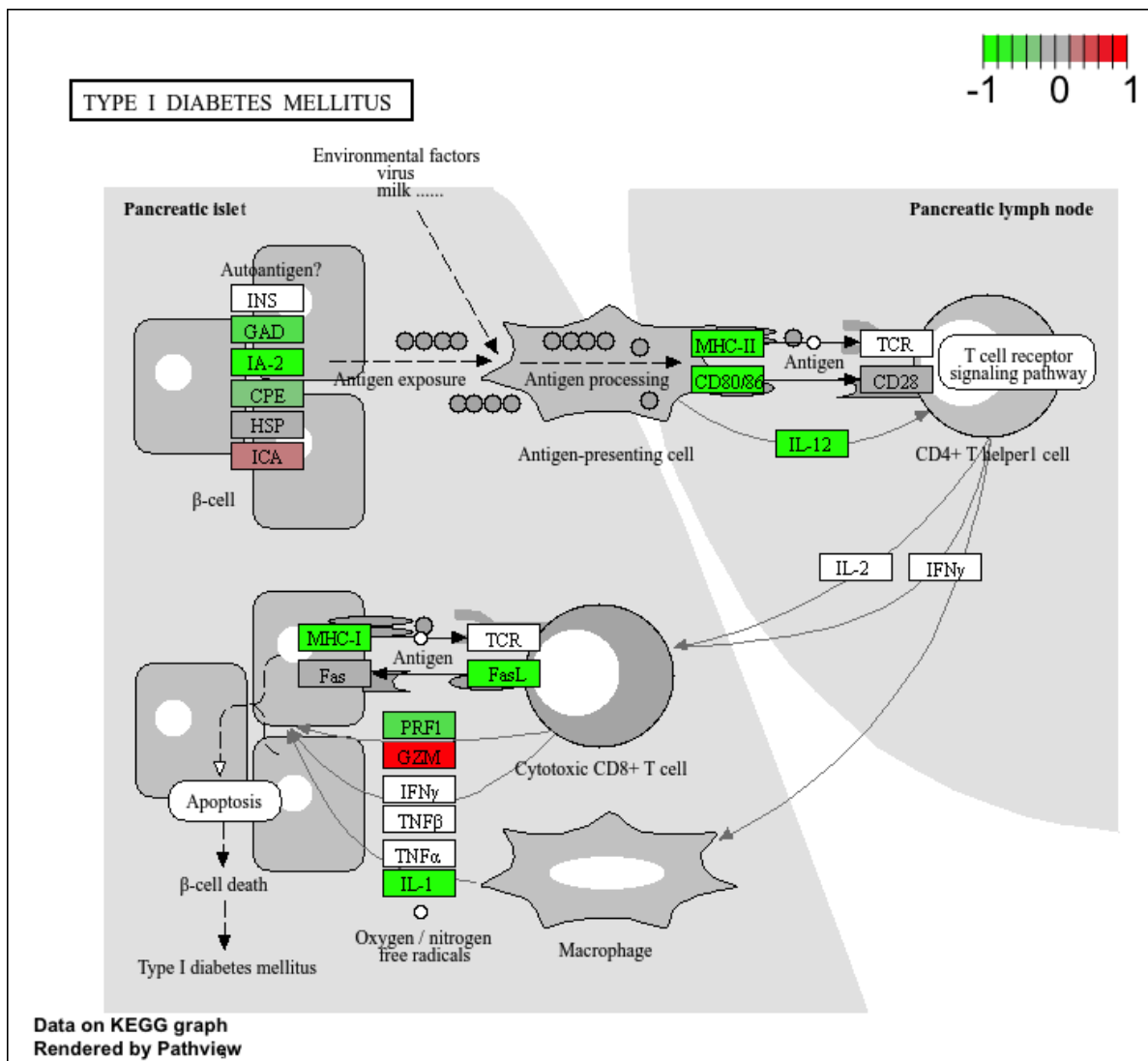
```
pathview(gene.data = foldchanges, pathway.id = top2.ids[2])
```

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory /Users/kaifyenepoya/Desktop/BIMM143/LAB_13

Info: Writing image file hsa04940.pathview.png





The top two down-regulated pathways were plotted using the pathway IDs from the GAGE results.

Optional: Plotting Counts for CRISPLD2

Find the CRISPLD2 gene.

```
i <- grep("CRISPLD2", res$symbol)
res[i, ]
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 1 row and 10 columns

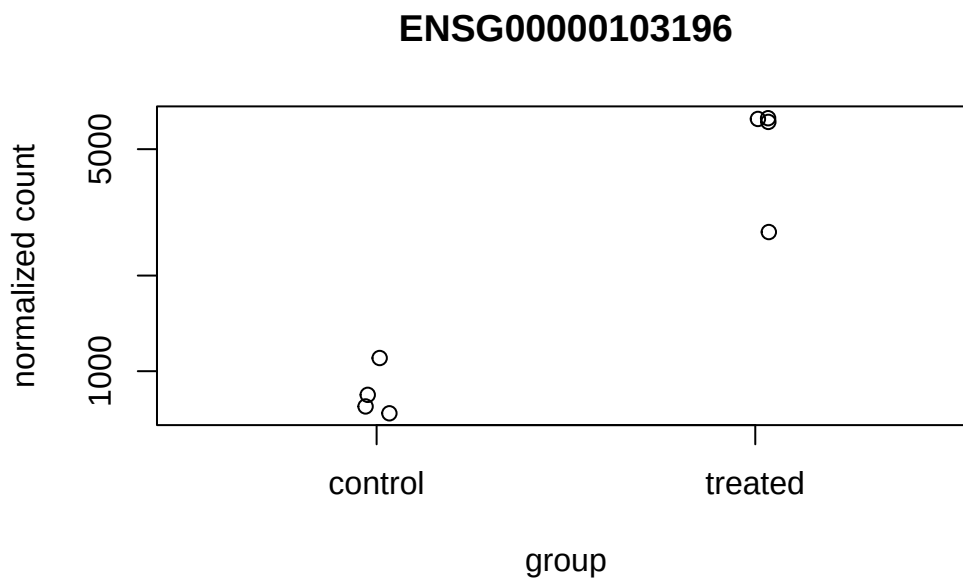
	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG00000103196	3096.16	2.62603	0.267454	9.81865	9.35839e-23
	padj	symbol	entrez	uniprot	
	<numeric>	<character>	<character>	<character>	
ENSG00000103196	3.37459e-20	CRISPLD2	83716	AOA140VK80	
		genename			
		<character>			
ENSG00000103196		cysteine rich secret..			

```
rownames(res[i, ])
```

```
[1] "ENSG00000103196"
```

Plot counts for CRISPLD2.

```
plotCounts(dds, gene = "ENSG00000103196", intgroup = "dex")
```

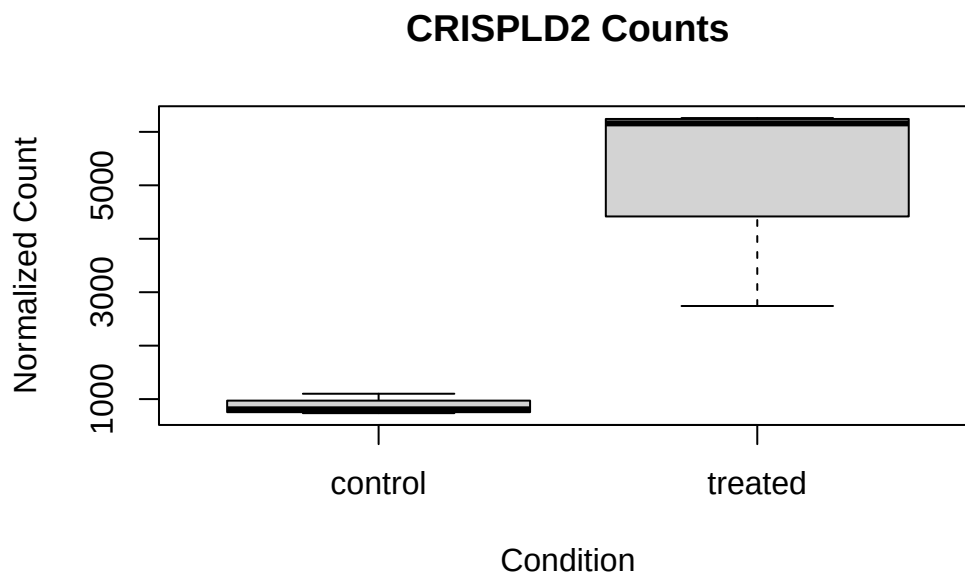


Return the data and make a boxplot.

```
d <- plotCounts(dds, gene = "ENSG00000103196", intgroup = "dex", returnData = TRUE)
head(d)
```

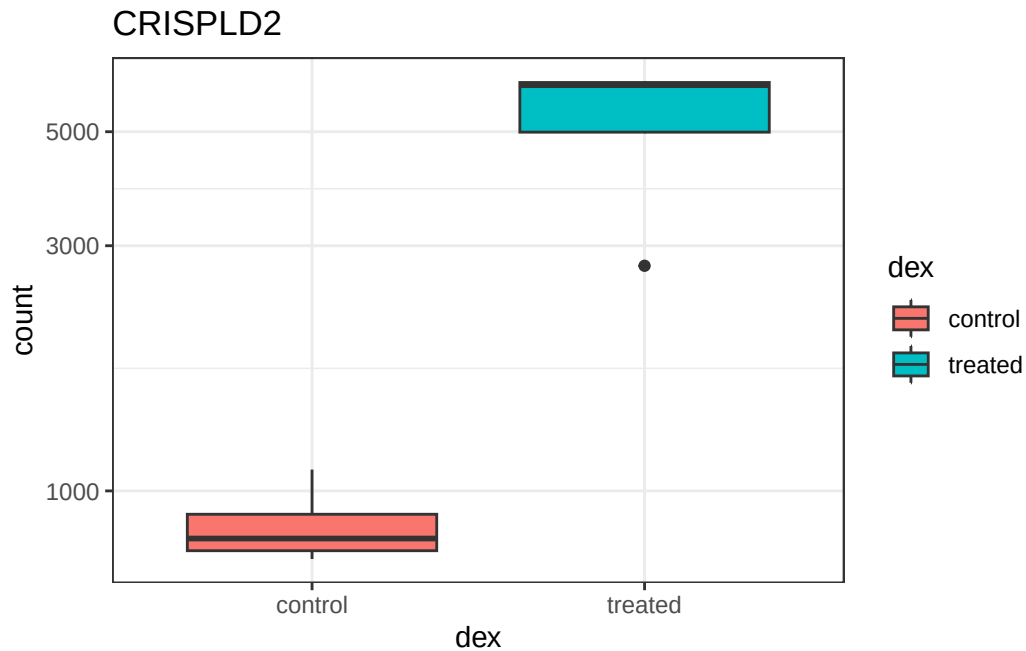
	count	dex
SRR1039508	774.5002	control
SRR1039509	6258.7915	treated
SRR1039512	1100.2741	control
SRR1039513	6093.0324	treated
SRR1039516	736.9483	control
SRR1039517	2742.1908	treated

```
boxplot(count ~ dex, data = d,
        xlab = "Condition",
        ylab = "Normalized Count",
        main = "CRISPLD2 Counts")
```



Make a ggplot version.

```
ggplot(d, aes(dex, count, fill = dex)) +
  geom_boxplot() +
  scale_y_log10() +
  ggtitle("CRISPLD2") +
  theme_bw()
```



I prefer the ggplot version because it looks cleaner, and the log scale makes the expression differences easier to compare.

Session Information

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Sonoma 14.4.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/Los_Angeles
```

```
tzcode source: internal
```

attached base packages:

```
[1] stats4      stats      graphics  grDevices  utils      datasets  methods
[8] base
```

other attached packages:

```
[1] gageData_2.48.0      gage_2.60.0
[3] pathview_1.50.0      org.Hs.eg.db_3.22.0
[5] AnnotationDbi_1.72.0 DESeq2_1.50.2
[7] SummarizedExperiment_1.40.0 Biobase_2.70.0
[9] MatrixGenerics_1.22.0 matrixStats_1.5.0
[11] GenomicRanges_1.62.1 Seqinfo_1.0.0
[13] IRanges_2.44.0      S4Vectors_0.48.1
[15] BiocGenerics_0.56.0  generics_0.1.4
[17] ggplot2_4.0.2
```

loaded via a namespace (and not attached):

```
[1] KEGGREST_1.50.0      gtable_0.3.6      xfun_0.57
[4] lattice_0.22-9       bitops_1.0-9      vctrs_0.7.2
[7] tools_4.5.3          parallel_4.5.3    tibble_3.3.1
[10] RSQLite_3.52.0       blob_1.3.0        pkgconfig_2.0.3
[13] Matrix_1.7-4         RColorBrewer_1.1-3 S7_0.2.1
[16] graph_1.88.1         lifecycle_1.0.5   compiler_4.5.3
[19] farver_2.1.2         Biostrings_2.78.0 codetools_0.2-20
[22] htmltools_0.5.9      RCurl_1.98-1.18   yaml_2.3.12
[25] GO.db_3.22.0         pillar_1.11.1     crayon_1.5.3
[28] BiocParallel_1.44.0 DelayedArray_0.36.1 cachem_1.1.0
[31] abind_1.4-8          tidyselect_1.2.1  locfit_1.5-9.12
[34] digest_0.6.39        dplyr_1.2.1       labeling_0.4.3
[37] fastmap_1.2.0        grid_4.5.3        cli_3.6.5
[40] SparseArray_1.10.10 magrittr_2.0.5     S4Arrays_1.10.1
[43] XML_3.99-0.23        withr_3.0.2       scales_1.4.0
[46] bit64_4.8.0          rmarkdown_2.31    XVector_0.50.0
[49] httr_1.4.8           bit_4.6.0         png_0.1-9
[52] memoise_2.0.1        evaluate_1.0.5    knitr_1.51
[55] rlang_1.2.0          Rcpp_1.1.1        glue_1.8.0
[58] DBI_1.3.0            Rgraphviz_2.54.0  KEGGgraph_1.70.0
[61] jsonlite_2.0.0       R6_2.6.1
```